

1 Minimum Cost of Ropes

<https://www.geeksforgeeks.org/problems/minimum-cost-of-ropes-1587115620/>

Minimum Cost of Ropes

Given an array, `arr[]` of rope lengths, connect all ropes into a single rope with the minimum total cost. The cost to connect two ropes is the sum of their lengths.

Examples:

Input: `arr[] = [4, 3, 5]`

Output: 29

Explanation: First, connect 2 and 3 with a cost of 5, then connect 4 and 5 to a total of 14. The total cost is 5 + 14 = 19.

Input: `arr[] = [4, 2, 7, 6, 9]`

Output: 37

Explanation: First, connect ropes 4 and 2, which makes the array [6, 7, 6, 9]. Cost of this operation is 6. Then, connect ropes 6 and 7, which results in [12, 6, 9]. Cost of this operation is 12. Then, connect ropes 6 and 9, which results in [12, 15]. Cost of this operation is 15. Finally, connect ropes 12 and 15, which results in [27]. Total cost is 6 + 12 + 15 = 33.

Input: `arr[] = [10]`

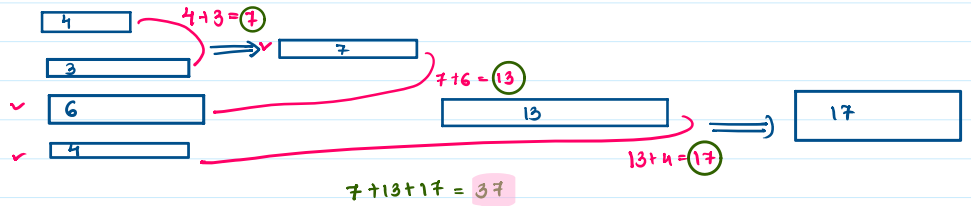
Output: 0

Explanation: Since there is only one rope, no connections are needed, so the cost is 0.

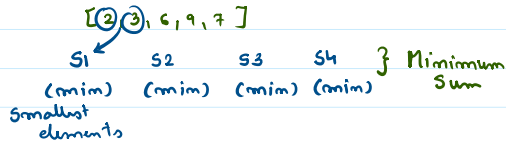
Constraints:

1 ≤ `arr.size()` ≤ 10⁵

1 ≤ `arr[i]` ≤ 10⁴



Solution :



$n=5$

$[2, 3, 6, 9, 7] \rightarrow [5, 6, 7, 9]$ ($2+3=5$)

$O(n \log n)$

$[5, 6, 7, 9] \rightarrow [11, 7, 9]$ ($5+6=11$)

$O(n \log n)$

$[11, 7, 9] \rightarrow [18, 9]$ ($11+7=18$)

$O(n \log n)$

$\vdots$

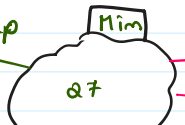
$O(n \log n)$

Smallest element is needed

$n \Rightarrow O(n^2 \log n)$

Using Priority Queue:

Ascending PQ



Stop

$[2, 3, 6, 9, 7]$

S1: $2+3=5$

2x remove

1x add

$\rightarrow O(3 \log n)$

S2: $5+6=11$

2x remove

1x add

$\rightarrow O(3 \log n)$

S3: $7+9=16$

2x remove

1x add

$\rightarrow O(3 \log n)$

S4: $11+16=27$

2x remove

1x add

$\rightarrow O(3 \log n)$

S5: Only one element in PQ

$\rightarrow O(3 \log n)$

$n \Rightarrow O(3n \log n)$
 $O(n \log n)$

Operation of PQ :

- ① `add()` $\rightarrow O(\log n)$
- ② `remove()` $\rightarrow O(\log n)$
- ③ `getMin()` OR `getMax()` $\rightarrow O(1)$
- ④ `size()` $\rightarrow O(1)$

" Removing a specific element from PQ is difficult "

2 Design Min-Max Queue :

<https://www.geeksforgeeks.org/problems/design-minmax-queue/>

Design minmax queue

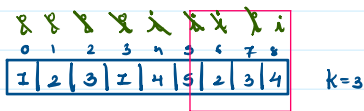
Design a **SpecialQueue** data structure that functions like a normal queue but with additional features. The **SpecialQueue** must support the following operations:

- `enqueue(x)`: Insert an element `x` at the rear of the queue.
- `dequeue()`: Remove the element from the front of the queue.
- `getFront()`: Return the front element without removing.
- `getMin()`: Return the minimum element in the queue in $O(1)$ time.
- `getMax()`: Return the maximum element in the queue in $O(1)$ time.

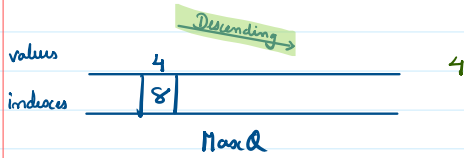
There will be a sequence of queries `queries[]`. The queries are represented in numeric form:

- 1: `enqueue(x)`
- 2: `dequeue()`
- 3: `getFront()`
- 4: `getMin()`
- 5: `getMax()`

Explanation: When $k = 1$, each element in the array is its own subarray, so the output is simply the same array
Constraints:
 $1 \leq \text{arr.size}() \leq 10^4$
 $1 \leq k \leq \text{arr.size}()$
 $0 \leq \text{arr}[i] \leq 10^4$



[1, 2, 3, 1, 4, 5, 2, 3, 4] $k = 3$



size = k
start = $i - k + 1$
ending = i

OIP
3 3 4 5 5 5 4

Max element
 $[s = i - k + 1, e = i]$
[6, 8]
↑ ↑
8

Break till
8:35

4 First Negative in every window of size k:

<https://www.geeksforgeeks.org/problems/first-negative-integer-in-every-window-of-size-k3345/1>

<https://leetcode.com/problems/first-negative-integer-in-every-window-of-size-k/>

first negative in every window of size k

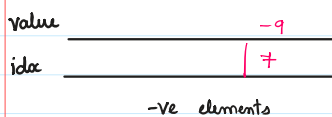
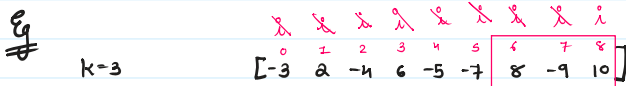
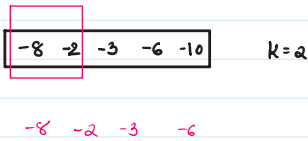
Given an array `arr[]` and a positive integer `k`, find the first negative integer for each and every window (contiguous subarray) of size `k`.

Note: If a window does not contain a negative integer, then return 0 for that window.

Examples:
Input: `arr[] = [-8, 2, 3, -6, 10]`, `k = 2`
Output: `[-8, 0, -6, -6]`
Explanation:
Window `[-8, 2]`: first negative integer is -8.
Window `[2, 3]`: no negative integers, output is 0.
Window `[3, -6]`: first negative integer is -6.
Window `[-6, 10]`: first negative integer is -6.
Input: `arr[] = [12, -1, -7, 8, -15, 30, 16, 28]`, `k = 3`
Output: `[-1, -1, -7, -15, -15, 0]`
Explanation:
Window `[12, -1, -7]`: first negative integer is -1.
Window `[-1, -7, 8]`: first negative integer is -1.
Window `[-7, 8, -15]`: first negative integer is -7.
Window `[8, -15, 30]`: first negative integer is -15.
Window `[-15, 30, 16]`: first negative integer is -15.
Window `[30, 16, 28]`: no negative integers, output is 0.

Input: `arr[] = [12, 1, 3, 5]`, `k = 3`
Output: `[0, 0]`
Explanation:
Window `[12, 1, 3]`: no negative integers, output is 0.
Window `[1, 3, 5]`: no negative integers, output is 0.

Constraints:
 $1 \leq \text{arr.size}() \leq 10^4$
 $-10^9 \leq \text{arr}[i] \leq 10^9$
 $1 \leq k \leq \text{arr.size}()$



First negative
 $[i - k + 1, i]$
[6, 8]
↑ ↑
7

OIP

-ve elements

[6, 8]

7

O/P

-3 -4 -4 -5 -5 -7 -9